BEAM AFFILIATE

Technical and Operational Project Documentation

Architecture, product scope, data model, APIs, configuration, deployment, security, testing, and maintenance.

Document version 1.0

Application version 1.1.0

Prepared 21 July 2026

Classification Internal project documentation

Contents

1. Executive Summary

Beam Affiliate is a full-stack affiliate marketing and partner-management platform. It gives administrators one place to operate affiliate programs, products, partners, referrals, commissions, payouts, transactions, reports, coupons, invoices, email campaigns, integrations, API keys, and team access. Affiliates receive a dedicated portal for referral links, product promotion, referrals, commissions, transactions, payouts, resources, reports, and profile management.

The application is implemented as a Next.js App Router project with TypeScript and React. PostgreSQL is the system of record and Prisma provides the data-access layer. Authentication uses signed JSON Web Tokens stored in an authentication cookie, with role-based access checks for administrator and affiliate areas. Stripe, Beam Wallet, bank-transfer webhooks, Resend email, push notifications, and outbound webhooks provide external integration points.

The codebase currently includes:

- 20 administrator pages and 12 affiliate pages.
- 73 API route files grouped by business domain.
- 31 Prisma data models.
- Multi-stage Docker packaging and a Docker Compose development stack.
- Health checks, audit records, API usage logs, rate-limit records, webhook logs, and email delivery logs.

This document describes the implemented system. Environment-specific credentials, legal policies, production infrastructure status, and operational ownership must be confirmed separately before a production launch.

2. Product Scope

2.1 Administrator capabilities

Administrators can:

- View program, revenue, conversion, commission, and partner analytics.
- Create and manage affiliate partners, partner groups, and commission overrides.
- Review referrals and record their status and review notes.
- Manage products, product images, categories, prices, and display order.
- Review transactions, refunds, payment proofs, and checkout activity.
- Approve, mature, claw back, and pay commissions.
- Create payouts and track processing status.
- Create programs, coupons, resources, invoices, reports, and scheduled reports.
- Create and test email templates and campaigns.
- Configure integration settings, API keys, webhooks, and API usage controls.
- Invite and manage team members with role and permission metadata.
- Configure branding, payout policy, tracking behavior, and program settings.

2.2 Affiliate capabilities

Affiliates can:

- Review earnings, activity, and performance from a personal dashboard.

- View and promote products using product-specific referral links.
- Generate and copy referral codes and links.
- Submit and track referrals.
- Review commissions and transaction history.
- Request and monitor payouts.
- Download marketing resources.
- View reports and progression/level information.
- Maintain profile and payout details.

2.3 Public and customer-facing capabilities

Public flows include:

- Registration, login, logout, OTP delivery, and OTP verification.
- Referral redirects and click attribution.
- Public product retrieval.
- Checkout creation and payment verification.
- Manual payment-proof submission.
- Conversion tracking and inbound webhooks.
- Health and API-documentation endpoints.

3. Technology Stack

3.1 Application

Layer	Technology	Purpose
Web framework	Next.js 16	App Router pages, server routes, middleware-compatible proxy, and standalone builds
UI runtime	React 19	Component-based administrator, affiliate, and public interfaces
Language	TypeScript 5	Static types across application and API code
Styling	Tailwind CSS 3 and Radix UI	Responsive design and accessible UI primitives
Forms and validation	React Hook Form, Zod	Form state and request/input validation
Charts	Recharts	Dashboard analytics and reports
Animation	Framer Motion	UI transitions

3.2 Data and back-end services

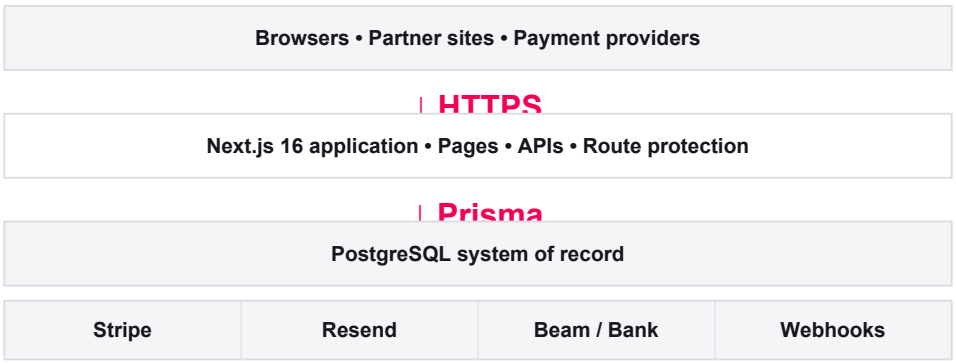
Component	Technology	Purpose
Primary database	PostgreSQL	Durable relational data store
ORM	Prisma 6	Schema definition, generated client, queries, and relationships
Password security	bcryptjs	Password hashing with cost factor 12 in the registration flow
Token security	jose	JWT signing and verification
Payments	Stripe	Checkout, payment verification, and webhook processing
Email	Resend / Nodemailer	Transactional email and templates
External payment validation	Beam Wallet and bank APIs	Payment status and webhook validation

3.3 Packaging and operations

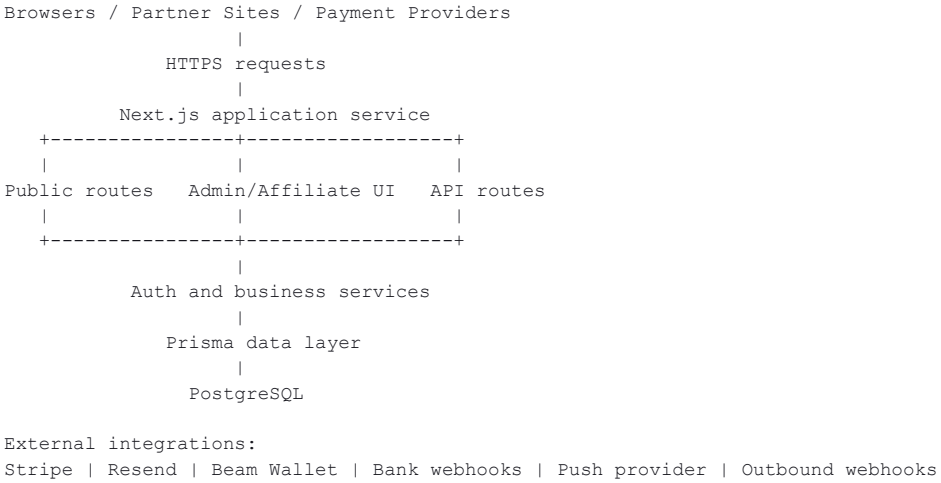
- `next.config.js` enables standalone Next.js output for container deployments.
- `Dockerfile` uses a multi-stage Node 20 Alpine image and runs the service as a non-root user.
- `docker-compose.yml` provides an application container and PostgreSQL 16 development database.
- `vercel.json` supports Vercel deployment.
- `/health` checks database connectivity and returns HTTP 503 when the database is unavailable.

> Version note: package metadata is authoritative for dependency versions. Some older narrative documents still mention Next.js 15 and should be updated to Next.js 16.

4. System Architecture



4.1 Logical architecture



4.2 Request processing

1. A browser or integration calls a page or API endpoint.
2. Requests under `/admin`, `/affiliate`, `/api/admin`, and `/api/affiliate` pass through `src/proxy.ts`.
3. The proxy reads the `auth-token` cookie, verifies the JWT with `JWT_SECRET`, and applies role checks.
4. Verified user ID and role values are forwarded in request headers.
5. Route handlers validate input, call service modules, and access PostgreSQL through Prisma.
6. Route handlers return JSON, redirect responses, or rendered Next.js pages.
7. Relevant actions can generate audit records, notifications, emails, commissions, or webhook activity.

4.3 Source layout

```
src/
  app/
    admin/              Administrator pages
    affiliate/          Affiliate pages
    api/                Server-side HTTP route handlers
    checkout/           Customer checkout flows
    login, register/    Authentication pages
    health/             Database-aware health endpoint
  components/           Product, notification, and reusable UI components
  hooks/                Client-side reusable hooks
  lib/                  Authentication and domain services
  proxy.ts              JWT and role-based route protection
prisma/
  schema.prisma         Relational data model
  seed.ts               Seed data
public/
  images/               Product and brand images
  scripts/              Browser-side tracking scripts
docs/                   Operational and feature documentation
```

5. User Roles and Access Control

5.1 Application roles

The primary `Role` enum contains:

- `ADMIN`: access to administrator pages and APIs.
- `AFFILIATE`: access to affiliate pages and APIs.

Team administration has a separate role model with `OWNER`, `ADMIN`, `MANAGER`, and `VIEWER`.

5.2 Authentication flow

1. Registration checks for an existing email address.
2. Passwords are hashed with bcrypt using 12 rounds.
3. Affiliate registration creates a linked affiliate profile, referral code, reseller ID, payout details, and zero starting balance.
4. Login verifies credentials and issues a signed JWT.
5. The token is stored in the `auth-token` cookie.
6. Protected requests are verified by `src/proxy.ts`.
7. Invalid, missing, or expired tokens produce HTTP 401 for APIs or a login redirect for pages.
8. Role mismatches produce HTTP 403 for APIs or a login redirect for pages.

5.3 Account state

User status supports `ACTIVE`, `INACTIVE`, `SUSPENDED`, and `PENDING`. The current registration service creates users as `ACTIVE`; this should be reviewed if administrator approval is required by business policy.

5.4 Security controls present in the model

- Unique user emails, referral codes, reseller IDs, coupon codes, invoice numbers, and API-key hashes.
- Hashed passwords and hashed API-key support.
- OTP expiry, use state, and attempt counters.
- API-key scopes, per-key rate limits, expiry, and usage logs.
- Persistent rate-limit windows.
- Webhook secrets, delivery status, attempt count, and retry scheduling.
- Audit logs linking actors, actions, object types, object IDs, and payloads.
- Role-based protected route groups.

6. Core Business Workflows

6.1 Affiliate onboarding

```
Registration -> User record -> Affiliate profile
             -> Referral code + reseller ID
             -> Admin notification -> Portal access
```

An affiliate profile is linked one-to-one with a user. It stores referral identity, payout details, account balance, optional partner group, and optional commission override.

6.2 Referral and attribution

1. An affiliate shares a referral link or product link.
2. A customer opens the referral route.
3. Tracking records referral context, IP address, user agent, referrer, and metadata.
4. A referral may be created or associated with later checkout activity.
5. A conversion records event type, amount, currency, state, and metadata.

Supported conversion event types are signup, purchase, trial, and lead.

6.3 Checkout and payment

The checkout subsystem supports:

- Product-based checkout creation.
- Stripe payment flows.
- Payment verification.
- Manual payment-proof submission.
- Beam Wallet and bank payment validation.
- Refund webhook processing.

Payment and webhook credentials must be environment-specific and must never be committed to source control.

6.4 Commission lifecycle

```
Conversion
   |
Commission calculation
   |
PENDING -> APPROVED -> PAID
   |          |
CANCELLED   CLAWBACK (when required)
```

Commission records preserve the applied rate, amount, affiliate, user, conversion, maturity date, approval metadata, payout link, and clawback note. Rules can be percentage-based or fixed. A partner-group rate or affiliate override can affect the effective rate.

6.5 Payout lifecycle

Payouts group one or more commissions and record amount, method, status, creator, processing time, and notes. Statuses are `PENDING`, `PROCESSING`, `COMPLETED`, and `FAILED`. Program settings carry payout frequency, minimum payout, hold period, and auto-approval policy.

6.6 Notifications and communications

The platform models:

- Email templates and delivery logs.
- Administrator and affiliate notifications.
- Push subscriptions.
- Outbound webhooks and delivery logs.
- Scheduled and emailed reports.

7. Data Model

7.1 Central financial relationship

```
User 1---0..1 Affiliate
              |
              +---* Referral ---* ReferralClick
              |        |
              |        +---* Transaction
              |
              +---* Conversion ---* Commission ---0..1 Payout
              |
              +---* Payout
```

7.2 Model groups

Domain	Models
Identity and access	User, Affiliate, OTP, TeamMember, ApiKey, ApiUsageLog, RateLimitEntry
Attribution	Referral, ReferralClick, Conversion
Finance	Commission, CommissionRule, Payout, Transaction, ManualPaymentProof, Invoice
Program management	Program, ProgramSettings, PartnerGroup, Product, Coupon, Resource
Communications	EmailTemplate, EmailLog, PushSubscription
Integrations and governance	IntegrationSettings, Webhook, WebhookLog, AuditLog
Reporting	ScheduledReport, SavedReport

7.3 Data conventions

- IDs use CUID strings.
- Monetary values are stored as integer minor units such as `amountCents`.
- Currency is stored alongside monetary records.
- Flexible configuration and event payloads use JSON columns.
- Database table and column names use snake case through Prisma mappings.
- Timestamp fields use creation/update defaults where applicable.
- Cascade deletion is used for several user, affiliate, referral, conversion, and API relationships.

7.4 Migration caution

The repository relies primarily on `prisma db push` in documented local workflows. Production environments should adopt reviewed, version-controlled Prisma migrations and run `prisma migrate deploy` through a controlled release step.

8. API Surface

The application contains 73 `route.ts` files. API access is organized by responsibility:

Prefix	Responsibility	Typical operations
`/api/auth`	Authentication	Login, logout, registration, session, OTP
`/api/admin`	Administration	CRUD and operational actions across partners, products, money, settings, reports, integrations, and team
`/api/affiliate`	Affiliate portal	Profile, links, referrals, transactions, commissions, payouts, resources, branding, gamification
`/api/track`	Attribution	Referral and conversion events
`/api/checkout`	Customer purchase	Create, verify, and submit payment proof
`/api/webhook`	Provider callbacks	Stripe, Beam Wallet, bank, conversion, and refund events
`/api/notifications`	Push notifications	Subscription and dispatch
`/api/products`	Public catalog	Active product retrieval
`/api/docs`	API discovery	API documentation payload

8.1 API design notes

- Route handlers export standard HTTP methods such as GET, POST, PUT, PATCH, and DELETE.
- Administrator and affiliate APIs are protected centrally by the proxy matcher.
- API routes return HTTP 401 for missing/invalid sessions and HTTP 403 for role violations.
- Financial and integration routes should additionally verify ownership, state transitions, idempotency, and provider signatures inside each handler.

9. Configuration

Copy `.env.example` to `.env.local` for local development. Never commit real credentials.

9.1 Required core variables

Variable	Purpose
`DATABASE_URL`	PostgreSQL connection string
`JWT_SECRET`	JWT signing and verification; generate at least 32 random bytes
`NEXT_PUBLIC_APP_URL`	Public application origin

9.2 Email

Variable	Purpose
`RESEND_API_KEY`	Resend service credential
`RESEND_FROM_EMAIL`	Verified sender identity
`ADMIN_EMAILS`	Comma-separated operational recipients

9.3 Payments and webhooks

Variable	Purpose
`STRIPE_SECRET_KEY`	Server-side Stripe API credential
`STRIPE_PUBLISHABLE_KEY`	Client-visible Stripe key
`STRIPE_WEBHOOK_SECRET`	Stripe event signature verification
`BEAM_WALLET_API_URL`	Beam Wallet API endpoint
`BEAM_WALLET_API_KEY`	Beam Wallet API credential
`BEAM_WALLET_MERCHANT_ID`	Beam Wallet merchant identity
`BEAM_WEBHOOK_SECRET`	Beam webhook verification
`BANK_WEBHOOK_SECRET`	Bank webhook verification

9.4 Automation and support

Variable	Purpose
`CRON_SECRET`	Authorizes scheduled internal operations
`PUSH_PROVIDER_URL`	Optional push provider endpoint
`SUPPORT_EMAIL`	User-facing support address

9.5 Secret-management rules

- Store production values in the deployment platform’s encrypted secret manager.
- Use separate credentials for local, test, staging, and production.
- Rotate keys after suspected exposure.
- Do not place secrets in Docker images, browser bundles, logs, screenshots, or documentation.
- Mark only intentionally public values with `NEXT_PUBLIC_`.

10. Local Development

10.1 Prerequisites

- Node.js 20 or a newer supported LTS release.
- npm.
- PostgreSQL 14+ (PostgreSQL 16 is used by Docker Compose).

10.2 Native setup

```
git clone <repository-url>
cd <repository-directory>
npm ci
cp .env.example .env.local
# Edit .env.local with development-only values.
npm run db:generate
npm run db:push
npm run db:seed
npm run dev
```

The development server runs at `http://localhost:3000` by default. The project explicitly uses the webpack development mode.

10.3 Available npm commands

Command	Purpose
`npm run dev`	Start the development server with webpack
`npm run build`	Generate Prisma Client and build Next.js
`npm start`	Start the production Next.js server
`npm run lint`	Run configured lint checks
`npm run db:generate`	Generate Prisma Client
`npm run db:push`	Synchronize schema directly to a development database
`npm run db:seed`	Seed development data
`npm run test:email`	Run the email test script
`npm run test:invite-url`	Verify invite URL behavior

10.4 Docker Compose setup

```
export JWT_SECRET="$(openssl rand -base64 32)"
docker compose up --build
```

This starts:

- `beam-app` on port 3000.
- PostgreSQL 16 on port 5432.
- A persistent `postgres_data` volume.

The default Compose database password is development-only and must not be used in production.

11. Build and Deployment

11.1 Production build

```
npm ci
npm run build
npm start
```

The build generates Prisma Client before invoking the Next.js compiler.

11.2 Container image

The Dockerfile:

1. Installs dependencies in a dedicated stage.
2. Generates Prisma Client.
3. Builds standalone Next.js output.
4. Copies only runtime assets into the final image.
5. Runs under the non-root `nextjs` user.
6. Exposes port 3000 and launches `server.js`.

Example:

```
docker build -t beam-affiliate:1.1.0 .
docker run --rm -p 3000:3000 --env-file .env.production beam-affiliate:1.1.0
```

11.3 Vercel

For Vercel:

1. Import the Git repository.
2. configure all required environment variables.
3. Connect a production PostgreSQL database.
4. Run schema migrations through a controlled job.
5. Deploy and verify `/health`.

Do not rely on the deployment filesystem for persistent user uploads. Use object storage for production-uploaded product images and documents.

11.4 Deployment verification

After deployment:

1. `GET /health` returns HTTP 200 with `status: "ok"` and `database: "ok"`.
2. Administrator login and dashboard load.
3. Affiliate login, products, and referral links load.
4. A test referral and conversion are attributed correctly.
5. Test-mode checkout completes and webhook processing is idempotent.
6. Commission creation and payout eligibility are correct.
7. Email delivery and webhook logs show successful processing.
8. Unauthorized access checks return HTTP 401 or 403 as expected.

12. Observability and Operations

12.1 Health monitoring

`GET /health` performs `SELECT 1` through Prisma and returns:

- Service name.
- Application status.
- Database status.
- Timestamp.
- Request latency in milliseconds.

Database failure produces HTTP 503.

12.2 Operational records

The database supports:

- Audit logs for actor and object activity.
- API usage logs containing endpoint, method, response status, latency, IP, and user agent.
- Email logs containing delivery state and error details.
- Webhook logs containing attempts, status code, response, error, and retry time.
- Payment-proof review metadata.

12.3 Recommended alerts

- Health endpoint unavailable or returning HTTP 503.
- Elevated HTTP 5xx rate.
- Database connection or query failures.
- Stripe, Beam Wallet, or bank webhook signature failures.
- Webhook retry backlog or repeated delivery failures.
- Email failure/bounce rate above threshold.
- Payout failures or unusual payout volume.
- Authentication failure spikes.
- API rate-limit saturation.

12.4 Backup and recovery

- Use automated encrypted PostgreSQL backups.
- Define a recovery-point objective and recovery-time objective.
- Test point-in-time recovery in a non-production account.
- Export and version deployment configuration.
- Preserve object-storage data independently from database backups.
- Document rollback to the previous application image and database-compatible release.

13. Testing Strategy

13.1 Current automated checks

The repository provides lint, TypeScript build validation through Next.js, application build, email-test, and invite-URL scripts. A comprehensive unit/integration test framework is not declared in `package.json`.

13.2 Recommended test pyramid

Layer	Priority coverage
Unit	Commission math, currency conversion, validation, fraud rules, referral-code generation, payment state transitions
Integration	Authentication, protected APIs, Prisma transactions, webhook signature verification, idempotency, payout creation
End-to-end	Registration, login, referral attribution, checkout, commission creation, admin approval, payout
Security	Authorization matrix, cookie flags, CSRF exposure, injection, stored XSS, file uploads, API-key scope enforcement
Operational	Container startup, migration, health check, backup restore, rollback

13.3 Minimum release gates

```
npm ci
npm run lint
npx tsc --noEmit
npm run build
```

Add automated tests and require all release gates on merge requests before production deployment.

14. Security and Production Readiness

14.1 Controls to retain

- Strong password hashing.
- JWT signature verification.
- Central administrator/affiliate role checks.
- Webhook secrets and delivery logging.
- API-key hashing, scopes, expiry, and rate limits.
- Non-root production container.
- Database-aware health endpoint.
- Audit logging for privileged actions.

14.2 Items requiring production verification

- Ensure `JWT_SECRET` is high entropy and consistent across instances.
- Ensure auth cookies are `HttpOnly`, `Secure`, and use an appropriate `SameSite` policy.
- Require provider signatures on every production webhook.
- Enforce idempotency for payments, conversions, refunds, commissions, and payouts.
- Remove or protect test-only endpoints in production.
- Remove sensitive debug logging from authentication and request middleware.
- Restrict file type, file size, and storage location for uploads.
- Confirm authorization inside every financial route, not only route-prefix checks.
- Add CSRF protection where cookie-authenticated state-changing requests need it.
- Apply database migrations through reviewed artifacts.
- Configure security headers and a content security policy.
- Run dependency, secret, SAST, and container-image scanning in CI.
- Retain audit and financial records according to legal policy.

14.3 Known documentation/code differences

- README framework versions are older than `package.json`.
- README describes pending affiliate approval, while the registration service currently activates users immediately.
- The current local operational artifacts should be reconciled with the intended GitLab/Vercel/AWS deployment process.

These differences should be resolved before the documentation is treated as a contractual operating procedure.

14.4 Verified implementation caveats

The following findings are based on the current implementation and should be triaged before production:

- Beam Wallet and bank webhook handlers only verify signatures when the corresponding secret exists. Production startup or request handling should fail closed when these secrets are absent.
- Development OTP verification can accept a submitted code without normal production verification safeguards. Development bypass behavior must be impossible when `NODE_ENV=production`.

- OTP-send rate limiting is not currently active, increasing email-abuse and credential-attack risk.
- The generic API-key rate-limit helper is not consistently invoked by route handlers; each public integration route needs explicit scope and rate enforcement.
- Local product uploads use `public/uploads/products`. This storage is ephemeral on Vercel and containers and may be unwritable for the non-root production user.
- Refund processing depends on conversion metadata that may not be populated by the Stripe success path, potentially preventing commission clawback lookup.
- Some Beam Wallet auto-payout outcomes can be marked paid while the provider still reports processing. Final balance deduction should occur only after confirmed completion.
- Beam Wallet and bank completion write the conversion, commission, transaction, and audit records without one enclosing database transaction, allowing partial financial state after a failure.
- System-generated audit entries use synthetic actor IDs while the schema requires a valid `User` foreign key. Introduce a durable system actor or make the actor relationship explicitly optional.
- The repository has no executable unit, integration, or end-to-end test suite.
- No version-controlled Prisma migrations are present; direct `db push` is unsuitable as the production migration strategy.
- Docker Compose starts the application and database but does not automatically apply the schema or seed data.
- Vercel security headers do not automatically apply to standalone Docker deployments; equivalent headers must be configured in the application or ingress layer.
- Infrastructure documents may refer to `/api/health`, while the implemented endpoint is `/health`. Load balancer and uptime checks must use the implemented path.
- Production bank-account fallback values must not be hardcoded. Deployment should fail when required payment instructions are missing.

These items do not imply that the platform is unusable; they identify controls needed to make financial processing, authentication, and operations resilient under production failure and attack conditions.

15. Release Checklist

Before release

- □ Peer review and automated checks pass.
- □ Production database migration reviewed and tested.
- □ Environment variables configured in secret storage.
- □ Production webhook endpoints and secrets configured.
- □ Stripe uses the intended test/live mode.
- □ Email sender domain is verified.
- □ Admin accounts use strong credentials and limited access.
- □ Object storage is configured for persistent uploads.
- □ Monitoring, alerting, backup, and rollback are tested.
- □ Legal terms, privacy policy, cookie policy, and payout terms are approved.

After release

- ☐ Health check and database connectivity confirmed.
- ☐ Admin and affiliate smoke tests completed.
- ☐ Referral attribution and conversion flow verified.
- ☐ Payment webhook and refund behavior verified.
- ☐ Commission and payout calculations sampled.
- ☐ Logs checked for errors or secret leakage.
- ☐ Release version, operator, and verification evidence recorded.

16. Troubleshooting

Database connection errors

1. Confirm `DATABASE_URL` syntax and network access.
2. Verify PostgreSQL is running and accepting TLS settings expected by the provider.
3. Run `npx prisma generate`.
4. Test the connection using Prisma or `psql`.
5. Check connection limits and use a pooled connection URL for serverless environments.

Unauthorized or redirect loops

1. Confirm `JWT_SECRET` matches the value used to issue the token.
2. Inspect the `auth-token` cookie domain, expiry, Secure, and SameSite settings.
3. Confirm the user role is `ADMIN` or `AFFILIATE` as required.
4. Clear stale cookies and authenticate again.

Build failures

1. Use the supported Node version.
2. Run `npm ci` from a clean dependency state.
3. Verify all build-time environment variables.
4. Run `npx prisma generate`.
5. Run `npx tsc --noEmit` before `npm run build`.

Missing uploaded images after deployment

The local filesystem is ephemeral on many serverless and container platforms. Store persistent uploads in object storage and save durable URLs in the database.

Webhook failures

1. Validate raw-body signature handling.
2. Confirm the provider’s configured endpoint and secret.
3. Check webhook logs, status codes, attempts, and next retry time.
4. Ensure processing is idempotent before replaying an event.

17. Maintenance Guide

Routine

- Review dependency and security updates weekly.
- Review application errors and failed integrations daily.
- Reconcile commissions, payouts, and payment-provider reports on a defined schedule.
- Rotate credentials and review privileged access periodically.
- Test database restoration and deployment rollback regularly.

When changing the schema

1. Update `prisma/schema.prisma`.
2. Create and review a migration.
3. Test migration and rollback against representative data.
4. Regenerate Prisma Client.
5. Update API types and documentation.
6. Deploy the migration before or with compatible application code.

When adding an API

1. Define authentication and authorization requirements.
2. Validate all input and normalize outputs.
3. Add rate limits and idempotency where appropriate.
4. Add audit records for privileged or financial operations.
5. Add tests and update this document/API reference.

18. Reference Files

Area	Source of truth
Dependencies and scripts	`package.json`
Database model	`prisma/schema.prisma`
Authentication service	`src/lib/auth.ts`
Protected route policy	`src/proxy.ts`
API handlers	`src/app/api/**/route.ts`
Administrator UI	`src/app/admin/**`
Affiliate UI	`src/app/affiliate/**`
Environment template	`.env.example`
Container build	`Dockerfile`
Local containers	`docker-compose.yml`
Next.js build configuration	`next.config.js`
Health endpoint	`src/app/health/route.ts`

Area	Source of truth
Data seed	`prisma/seed.ts`

19. Glossary

Term	Meaning
Affiliate	Partner who promotes products or services and earns commission
Referral	A prospective customer associated with an affiliate
Conversion	A tracked business event such as a lead, signup, trial, or purchase
Commission	Affiliate earnings derived from an approved conversion
Payout	A grouped transfer of payable commissions to an affiliate
Reseller ID	Unique external identity assigned to an affiliate
Cookie duration	Attribution window used for referral tracking
Clawback	Reversal of commission due to refund, fraud, or policy
Idempotency	Guarantee that replaying the same event does not duplicate its effects

20. Document Control

This document was generated from the repository state available on 21 July 2026. It intentionally excludes real credentials and customer data. Validate the document against the target release whenever dependencies, database models, business rules, APIs, authentication, or deployment architecture change.